Project Interim Report

On

File Compression Simulator

(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming With Java

SESSION: 2025-26

Submitted By:

RItesh kumar

mahto

RU-25-11179

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with Google

Under the Guidance of

Mr. Ashish Shivare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

Abstract

The School Administration Management System is a Java-based console application developed using core Object-Oriented Programming (OOP) concepts such as inheritance and polymorphism. The system is needed because educational institutions require structured and efficient ways to manage different roles, including teachers and students, in a unified environment. The project defines a base class *Person* with common attributes and extends it into specialized subclasses *Teacher* and *Student*, each overriding key methods to provide role-specific behavior. Through runtime polymorphism, the system dynamically dispatches the correct method at execution time, demonstrating how a single interface can represent multiple types of objects. The project illustrates clean, scalable class design and helps learners understand the practical application of OOP principles in real-world administrative scenarios.

Introduction

Managing an educational institution requires a systematic approach to handle diverse roles such as teachers and students. Traditionally, such management was done manually or with fragmented tools, leading to inefficiencies and errors. This project addresses that challenge by building a School Administration Management System using Java.

The system leverages OOP principles to model real-world entities. A base class *Person* encapsulates shared attributes like name and age. Specialized subclasses *Teacher* and *Student* inherit from *Person* and introduce role-specific attributes such as subject and salary for teachers, and roll number and course for students. The system demonstrates how a *Person[]* array can hold different object types and invoke the correct overridden *showRole()* method at runtime, which is the essence of dynamic method dispatch.

Key features of this project include:

- Role-based display of information for teachers and students
- Code reuse through a well-structured inheritance hierarchy
- Runtime polymorphism using a Person array
- Modular and readable class design

Objectives

- Define and manage Teacher and Student records under a common Person hierarchy
- Demonstrate method overriding with role-specific showRole() behavior
- Implement runtime polymorphism using a Person[] array
- Apply core OOP concepts: inheritance, encapsulation, and polymorphism
- Produce clean, readable, and reusable Java code

Scope

This project is suitable for educational purposes and beginner-level Java learners. It can be extended in future with a graphical user interface, database integration for persistent storage of

records, additional roles such as Administrator or Staff, and a search/update functionality for existing records.

System Requirements

Hardware: Computer with 4GB RAM

Software: Java JDK 8 or above, IDE (VS Code / Eclipse / Edit Plus)

Methodology

The development followed a step-by-step class-design approach. First, a base class *Person* was created with common attributes (name and age) and a general *showRole()* method. Next, a *Teacher* subclass was created extending *Person*, adding subject and salary attributes, and overriding *showRole()*. Similarly, a *Student* subclass was created with roll number and course attributes, also overriding *showRole()*. Finally, a main class was implemented to instantiate both subclasses into a *Person[]* array, iterate through it, and call *showRole()* on each object to demonstrate polymorphism.

Flowchart & Class Diagram

The flowchart starts with program initialization, moves to object creation (Teacher and Student), stores them in a Person array, loops through each element calling showRole(), and ends after displaying all outputs.

The class diagram is as follows:

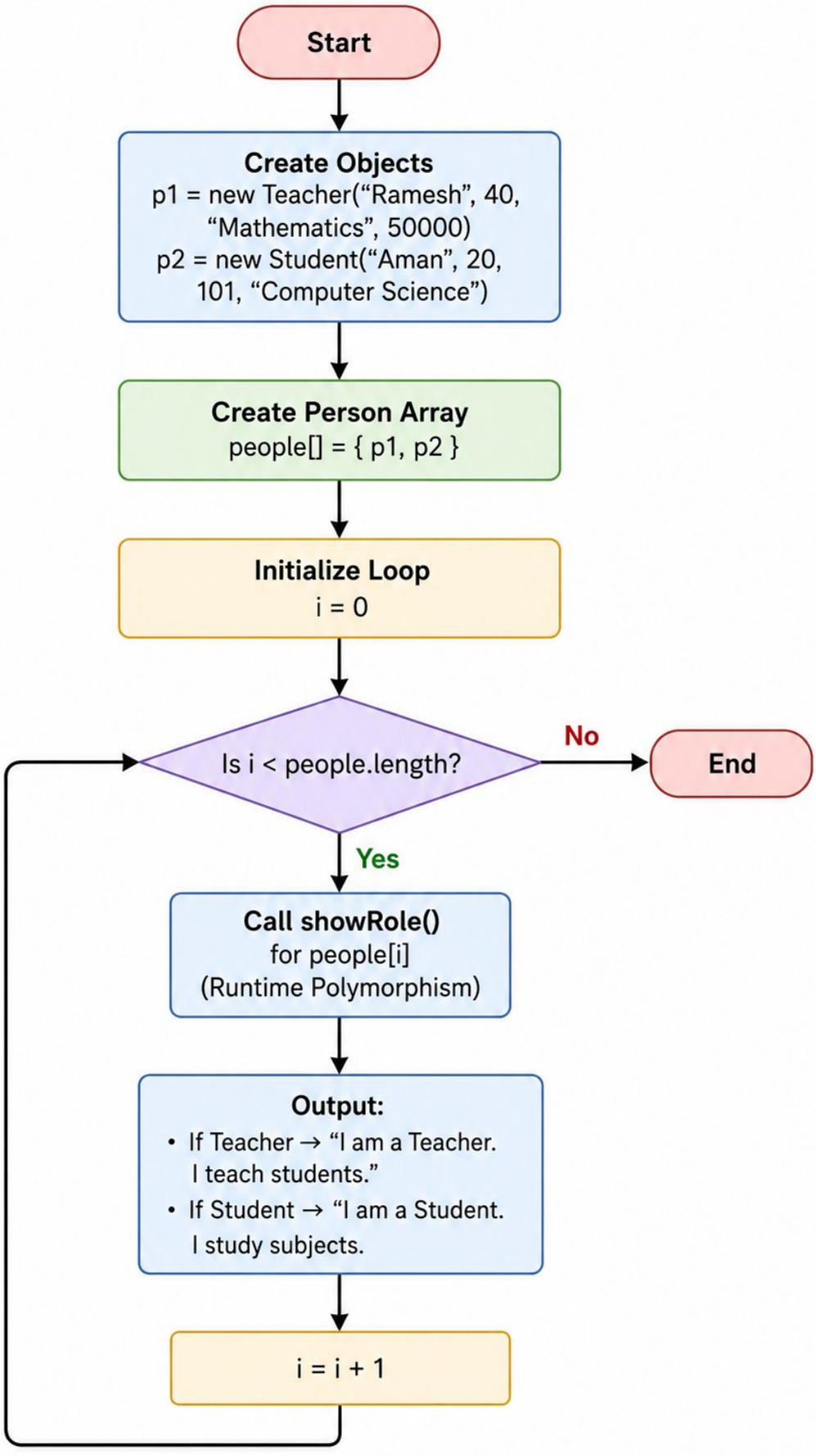
Person
- name : String - age : int
+ showRole() : void + getName() : String + getAge() : int

Subclasses inheriting from Person:

Teacher
- subject : String - salary : double
+ showRole() : void + getSubject() : String + getSalary() : double

Student
- rollNumber : int - course : String
+ showRole() : void + getRollNumber() : int + getCourse() : String

Flowchart: School Administration Management System



Implementation

The system consists of three main classes:

1. Person (Base Class): This class contains two attributes — *name* (String) and *age* (int). It defines a general *showRole()* method that prints a default message. It also includes getter methods for both attributes. This class serves as the parent for all role-based entities in the system.

2. Teacher (Subclass of Person): This class inherits *name* and *age* from *Person* and adds two new attributes: *subject* (String) and *salary* (double). The *showRole()* method is overridden to print a teacher-specific message. A parameterized constructor initializes all four attributes.

3. Student (Subclass of Person): This class inherits from *Person* and adds *rollNumber* (int) and *course* (String). The *showRole()* method is overridden to display a student-specific message. A constructor initializes all attributes including those inherited from the parent.

4. Main Class (SchoolAdmin): This class creates a *Person[]* array of size 2. Index 0 holds a *Teacher* object and index 1 holds a *Student* object. A for-each loop iterates through the array and calls *showRole()* on each element. Java's dynamic method dispatch ensures the correct overridden version is called at runtime.

Sample Input / Output

The following is the expected console output when the program is executed:

```
I am a Teacher. I teach students.
I am a Student. I study subjects.
```

Future Enhancements

- Add a database backend (MySQL/SQLite) for persistent storage of records
- Develop a graphical user interface (GUI) using JavaFX or Swing
- Introduce additional roles such as Admin, Principal, or Non-Teaching Staff
- Implement search, update, and delete operations for teacher and student records
- Add authentication and login functionality for different user roles

Gantt Chart

1. Planning & Structure

A Gantt chart helps break the project into tasks (design, coding, testing, etc.) and set timelines. Even if the project is built once, planning ensures each phase is completed systematically.

2. Time Management

It shows:

- When each task starts

- When it should finish
- This helps avoid delays and last-minute panic

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis	■■■■■			
Class Design	■■■■■	■■■■■		
Coding		■■■■■	■■■■■	
Testing & Debugging			■■■■■	
Report Writing			■■■■■	■■■■■
Final Submission				■■■■■

Conclusion

The School Administration Management System successfully demonstrates the application of core Java OOP concepts including inheritance, method overriding, and runtime polymorphism. By modeling real-world entities like Teacher and Student as subclasses of a common Person base class, the project achieves code reusability and clean modular design. The use of a Person[] array to hold different object types and dynamically invoke the correct showRole() method at runtime effectively illustrates dynamic method dispatch. This project provides a strong foundation for building more complex, scalable educational management systems in the future.

References

Book

Format: [1] Author(s), *Book Title*, Edition. City: Publisher, Year.

1 H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

Website

Format: [2] Author (if any), "Title of page," Website Name. [Online]. Available: URL

2 Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com

Journal Article

Format: [3] Author(s), "Title of paper," Journal Name, vol., no., pp., year.

3 A. Sharma and R. Gupta, "Object-Oriented Design Patterns," *International Journal of Software Engineering*, vol. 3, no. 1, pp. 22-30, 2019.

Conference Paper

Format: [4] Author(s), "Paper title," in Proc. Conference Name, year, pp.

4 S. Kumar, "OOP in Educational Management Systems," in Proc. IEEE Conf. on Software Engineering, 2022, pp. 88-93.